

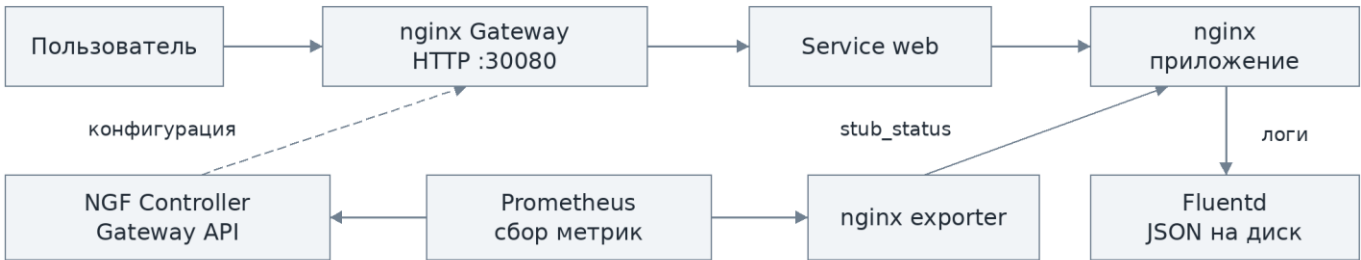
Паспорт DevOps решения

MTC ENGINEER HACK. Kubernetes и Gateway API на Ubuntu 24.04.

Собран одноузловой Kubernetes-стенд. Приложение на nginx доступно через Gateway API, Prometheus получает метрики, Fluentd собирает access- и error-логи. Развёртывание запускается .sh-скриптом, а проверка результата написана на Python с ujson 6.0.0 в отдельном venv.

Как устроено решение

Для Kubernetes выбрал kubeadm: он соответствует приоритету задания и позволяет настроить кластер на Ubuntu. Контейнеры запускает containerd, сеть Pod обеспечивает Flannel. Control plane и приложение работают на одном узле.



NGF Controller настраивает маршруты, nginx Gateway принимает HTTP. Prometheus развёрнут StatefulSet; Fluentd собирает логи рядом с nginx и сохраняет JSON на диске.

Закреплённые версии

Компонент	Версия в конфигурации
Kubernetes через kubeadm	1.35.9
Flannel	0.28.4
Gateway API	1.6.1
NGINX Gateway Fabric	2.7.2
nginx приложения	1.28.0-alpine
Prometheus	3.6.0
nginx exporter	1.4.2
Fluentd	v1.18.0-debian-1.0

Мой стенд: Ubuntu 24.04.5 LTS, WSL2; kubelet v1.35.9; runtime containerd://2.2.1. Фактические версии пакетов и digest образов собирает scripts/collect-versions.py.

Запуск

Нужна выделенная Ubuntu 24.04 amd64 с systemd и sudo. Рекомендуется 4 CPU, 8 GB RAM и 30 GB диска. Интернет нужен для apt и образов контейнеров. Для локальной демонстрации подходит WSL, для эксперта - VM с белым и статическим IP.

```
sudo bash deploy.sh
```

Функциональность сервисов

Приложение и Gateway API

nginx выбрал для простого проверяемого ответа: Hello World на / и JSON на /info. Для Gateway API созданы GatewayClass nginx, Gateway lab и три HTTPRoute. Маршрут web добавляет заголовок X-DevOps-Lab: gateway-api. По нему можно проверить, что запрос прошёл через Gateway.

Маршрут /api/info переписывается в /info через URLRewrite. /legacy возвращает HTTP 302 на /api/info, сохраняя внешний порт 30080. На /cats находится отдельная страница с изображением и подписью «Котик был доставлен через Kubernetes». Изображение включено в проект, внешняя загрузка не требуется.

```
curl -i http://NODE_IP:30080/
```

```
curl -i http://NODE_IP:30080/legacy
```

Мониторинг

Prometheus развёрнут отдельным StatefulSet с ConfigMap и hostPath: база сохраняется при обновлении Pod. Он получает метрики каждые 10 секунд; nginx exporter читает stub_status приложения. Проверяются доступность targets, nginx_up и пост nginx_http_requests_total после пяти запросов. Отчёт содержит значения до и после, а также состояние правил. Настроены два предупреждения и правила частоты запросов и памяти контроллера Gateway. Данные Prometheus сохраняются на hostPath; хранение блоков ограничено 7 днями и 1 GB.

```
kubectl -n devops-lab port-forward svc/prometheus 9090:9090
```

В браузере открыть <http://localhost:9090>. Targets должны быть UP, nginx_up - 1. Для роста счётчика сделать несколько запросов к приложению и дождаться следующего scrape.

Сбор логов

Fluentd работает рядом с nginx в одном Pod. Он читает access JSON и error text из общих файлов, добавляет source и записывает события в JSON на диск узла. Логи ротируются с атомарной заменой и USR1 при 8 MiB или через 10 секунд; остаются четыре архива. Fluentd следит за inode. Позиции и буфер сохраняются; CronJob удаляет старые собранные файлы.

Python-проверка отправляет запросы с уникальной меткой и ищет её в собранных access- и error-записях. Так проверяется сбор новых событий, а не наличие старого файла.

Полная проверка

```
/var/lib/devops-lab/venv/bin/python3 scripts/verify.py
```

Скрипт проверяет текущие статусы Gateway API, HTTP-ответы, rewrite, redirect, страницу /cats и байты PNG, рост метрик, правила Prometheus и оба вида логов. При ошибке возвращает ненулевой код завершения и обновляет отчёт на failed.

Проверка и развитие решения

Что проверено

03.10.2026 проверил работу стенда на WSL с ОС Ubuntu 24.04 и доступ к приложению из браузера Windows. Базовая проверка подтвердила Gateway, HTTP, метрики и оба вида логов Fluentd. Актуальный отчёт с маршрутами, /cats и ростом счётчика собирает scripts/verify.py.

Повторяемость проверяется scripts/check-repeat.sh: он выполняет два развёртывания и сохраняет отдельный отчёт для каждого запуска. Локально прошли Python-тесты, ShellCheck, проверка схем и конфигов

компонентов. GitHub Actions дважды проверяет стек в kind-кластере. Запуск CI после публикации и установка на свежей VM требуют отдельного подтверждения.

Главная особенность решения

Сильной стороной считаю проверку всей цепочки на Python: она отправляет HTTP-запросы, измеряет рост счётчика и находит новые записи Fluentd. Результат сохраняется в JSON; два последовательных запуска проверяют повторяемость развёртывания.

Автоматизация и качество

Версии закреплены в config/versions.json. Внешние манифесты сохранены в vendor с SHA-256. Для Pod заданы проверки состояния и ограничения ресурсов; основные контейнеры работают без root, с seccomp и файловой системой только для чтения. Приложению и мониторингу не выдаётся токен ServiceAccount.

В bootstrap исправил совместимость containerd 1.7/2.x, пустой DNS resolver в WSL и включение swap после загрузки. Исправил права /tmp для Fluentd и восстановление старого неготового Pod StatefulSet с проверкой владельца и UID. Конфиги web и Prometheus обновляются независимо, startupProbe даёт время на восстановление метрик.

Основная техническая сложность

Для логов выбирал между агентом на узле и Fluentd рядом с nginx. Выбрал sidecar с общей директорией: для одного приложения он проще и не требует доступа ко всем контейнерным логам узла. При нескольких репликах потребуется другая схема сбора.

Ограничения

Один узел и hostPath не обеспечивают HA и перенос данных. HTTP работает без TLS, NetworkPolicy не настроена. Логи ротируются в emptyDir 128 MiB; резкие всплески и отставание Fluentd требуют проверки.

Дальнейшее развитие

- Нагрузка - выполнить check-log-rotation.py на стенде и подобрать ресурсы по результату.
- TLS и уведомления - нужны сертификат, Secret и конфигурация Alertmanager.
- Несколько узлов - нужны PVC, резервные копии, централизованные логи и CNI с NetworkPolicy.
- Телеком-сценарий - добавить HTTP/TCP-проверки доступности и задержки; платные сервисы не нужны.